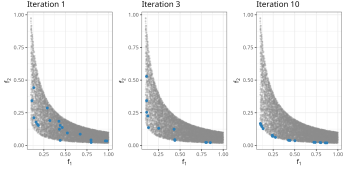
Optimization in Machine Learning

Multi-criteria Optimization
Evolutionary Approaches

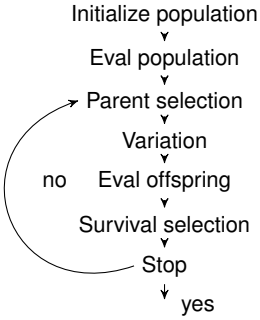


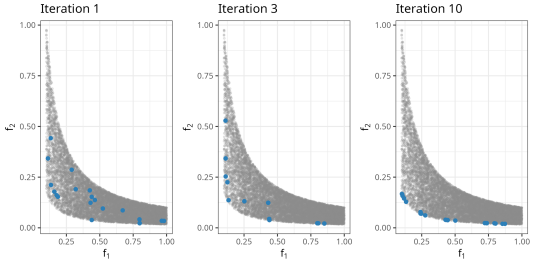
Learning goals

- EMOAs and population-based approach
- NSGA-II
- SMS-EMOA

A-POSTERIORI METHODS AND EVOLUTIONARY ALGORITHMS

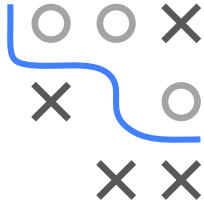
Evolutionary multi-objective algorithms (EMOAs) evolve a diverse population over time to approximate the Pareto front.





- Population-based search approximates a whole Pareto set in one run.
- The main steps (initialize, select, vary, survive, repeat) mirror standard EAs but must handle multiple objectives.

BASIC EA TEMPLATE LOOP

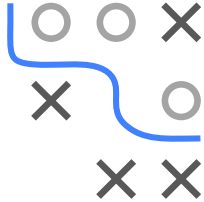


Algorithm Basic EA template loop

1. Initialize and evaluate population $\mathcal{P}_0 \subset \mathcal{S}$ with $|\mathcal{P}_0| = \mu$.
2. Set $t \leftarrow 0$.
3. Repeat until the stop criterion is fulfilled:
   1. Select parents from $\mathcal{P}_t$ and generate offspring $\mathcal{Q}_t$, with $|\mathcal{Q}_t| = \lambda$.
   2. Select μ survivors to form $\mathcal{P}_{t+1}$.
   3. Set $t \leftarrow t + 1$.

- As in standard EAs, we have parent selection, variation, and survival selection.
- For multi-objective EAs (EMOAs), ranking solutions by “fitness” is no longer straightforward. We need special selection strategies (non-dominated sorting, etc.).

NSGA-II



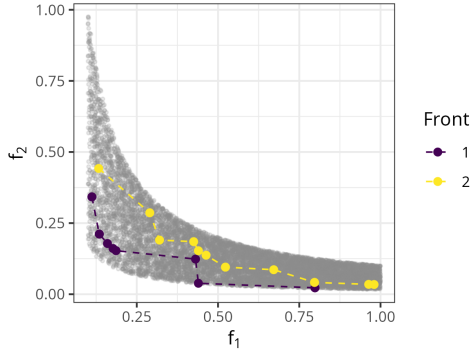
The **non-dominated sorting genetic algorithm (NSGA-II)** was published by ▸ Deb et al. 2002.

- Follows a $(\mu + \lambda)$ strategy for survival.
- Any standard variation operators (like polynomial mutation, SBX) can be used.
- Parent and survival selection both use:
  1. **Non-dominated sorting (NDS)** as the main ranking criterion,
  2. **Crowding distance** as a tie-break.

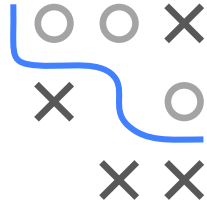
NSGA-II: NON-DOMINATED SORTING (I)

NDS partitions an objective space set into fronts $\mathcal{F}_1 \prec \mathcal{F}_2 \prec \mathcal{F}_3 \prec \ldots$.

- $\mathcal{F}_1$ is non-dominated, each $\boldsymbol{x} \in \mathcal{F}_2$ is dominated, but only by points in $\mathcal{F}_1$, each $\boldsymbol{x} \in \mathcal{F}_3$ is dominated, but only by points in $\mathcal{F}_1$ and $\mathcal{F}_2$, and so on.
- We can easily compute the partitioning by computing all non-dominated points $\mathcal{F}_1$, removing them, then computing the next layer of non-dominated points $\mathcal{F}_2$, and so on.

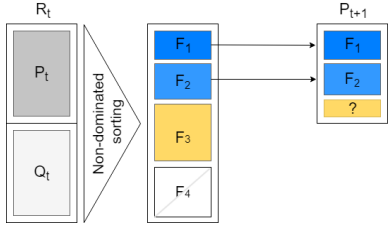


NSGA-II: NON-DOMINATED SORTING (II)



For **survival** in $(\mu + \lambda)$ selection:

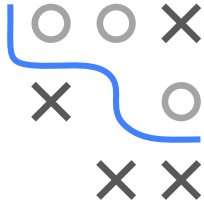
- We rank all $\mu + \lambda$ individuals by front index.
- We start filling the next population from $\mathcal{F}_1$ upward.
- If front $\mathcal{F}_i$ cannot fully fit (because we would exceed μ), we partially fill with a subset of $\mathcal{F}_i$.



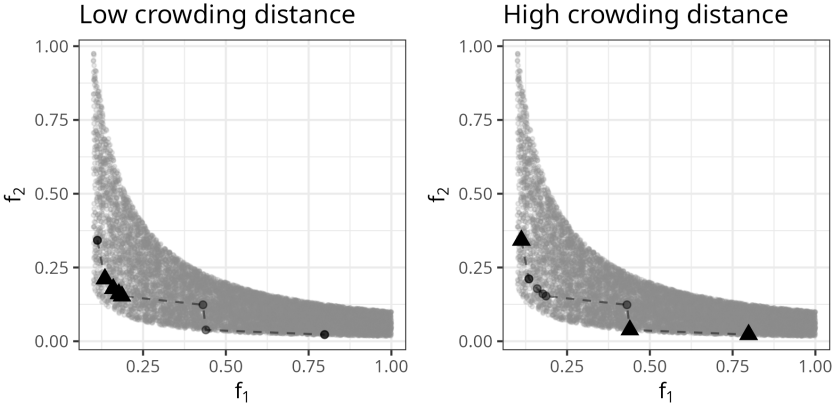
But: Which individuals survive from this partially included front? $\rightarrow$ *crowding distance* sorting.

(NDS is also used for parent selection with binary tournaments.)

NSGA-II: CROWDING DISTANCE (I)

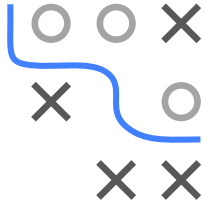


Idea: Prefer individuals that are more “spread out” (less crowded).



Left: not good, solutions bunched up. Right: better spread.

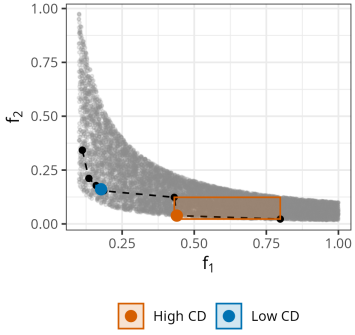
NSGA-II: CROWDING DISTANCE (II)



Crowding distance: For each objective f_j:

- Sort points by f_j (in ascending order).
- Normalize f_j values to [0, 1].
- "Border" points (lowest/highest f_j) get infinite distance (ensuring they always survive if possible).
- For internal points, distance is difference in f_j-values between next neighbors.

Then sum (or average) across all m objectives to get the final crowding score.



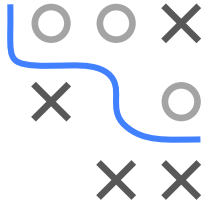
Red: High CD Blue: Low CD

HYPERVOLUME CONTRIBUTION

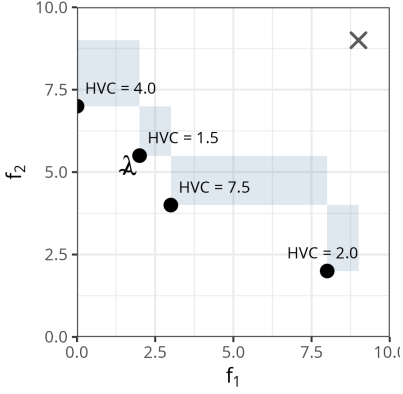
S-Metric-Selection uses the *hypervolume contribution* $\Delta s(\boldsymbol{x}, \mathcal{P})$ of each individual:

$$\Delta s(\boldsymbol{x}, \mathcal{P}) = S(\mathcal{P}, R) - S\big(\mathcal{P} \setminus \{\boldsymbol{x}\},\ R\big),$$

where $S(\cdot, R)$ is the dominated HV w.r.t. reference point R.



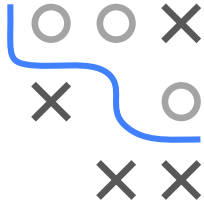
- Dark rectangles: unique HV part dominated only by that dot
- Gray cross: reference point R.
- The individual with the smallest HV contribution is “least important” for covering objective space.
- This is used by the SMS-EMOA to remove individuals in survival selection.



SMS-EMOA ALGORITHM

Algorithm SMS-EMOA

1. Generate initial population $\mathcal{P}_0$ of size μ.
2. Set $t \leftarrow 0$.
3. Repeat until termination:
   1. Create exactly one offspring $\mathbf{q}$ by recombination and mutation.
   2. Perform non-dominated sorting on $(\mathcal{P}_t \cup \{\mathbf{q}\})$.
   3. Let $\mathcal{F}_k$ be the last front.
   4. Set $\tilde{\boldsymbol{x}} \leftarrow \arg\min_{\boldsymbol{x} \in \mathcal{F}_k} \Delta s(\boldsymbol{x}, \mathcal{F}_k)$.
   5. Set $\mathcal{P}_{t+1} \leftarrow \big(\mathcal{P}_t \cup \{\mathbf{q}\}\big) \setminus \{\tilde{\boldsymbol{x}}\}$.
   6. Set $t \leftarrow t + 1$.



- As soon as we exceed size $\mu + 1$, remove the worst HV contributor from the *last front*.
- That way, each iteration keeps exactly μ individuals.
- Ranking is primarily by front membership; hypervolume contribution is the tiebreak.